Solidity Advance

By Code Eater

Contents

- Transfer vs Send Vs Call
- Calling Another Contract
- ABI encoding and Decoding
- Encoding Decoding Calldata
- Delegatecall
- Proxy Contract

Transfer vs Send Vs Call

```
function transferEther(address payable recipient) public {
    recipient.transfer(1 ether);
}
```

transfer

```
function sendEther(address payable recipient) public returns (bool) {
    bool sent = recipient.send(1 ether);
    return sent;
}
```

send

```
function callEther(address payable recipient) public returns (bool, bytes memory) {
(bool success, bytes memory data) = recipient.call{value: 1 ether}("");
return (success, data);
}
```

call

Transfer Vs Send Vs Call

Gas Forwarding - When a contract makes an external call (such as a function call to another contract), it needs to allocate a certain amount of gas for that call. This gas will be used to execute the called contract's code.

Method	Forwarded Gas	Return Value	Reverts on Failure	Use Case
`transfer`	2300	None	Yes	Simple and safe Ether transfer.However, it only forwards a fixed amount of gas (2300), which may not be enough for more complex operations in the receiving contract. This limitation has led to transfer being considered less safe, as it can lead to unexpected behavior if the receiving contract's fallback function consumes more than 2300 gas.
`send`	2300	Boolean	No	Handle failure manually. Same limitation as transfer.
`call`	Customizable	Boolean + data	No	Complex calls and function invocation.It allows you to forward all remaining gas or specify a certain amount. This flexibility makes it suitable for more complex operations. However, it also requires you to handle the possibility of reentrancy attacks and manually check the return value.

Calling Another Contract

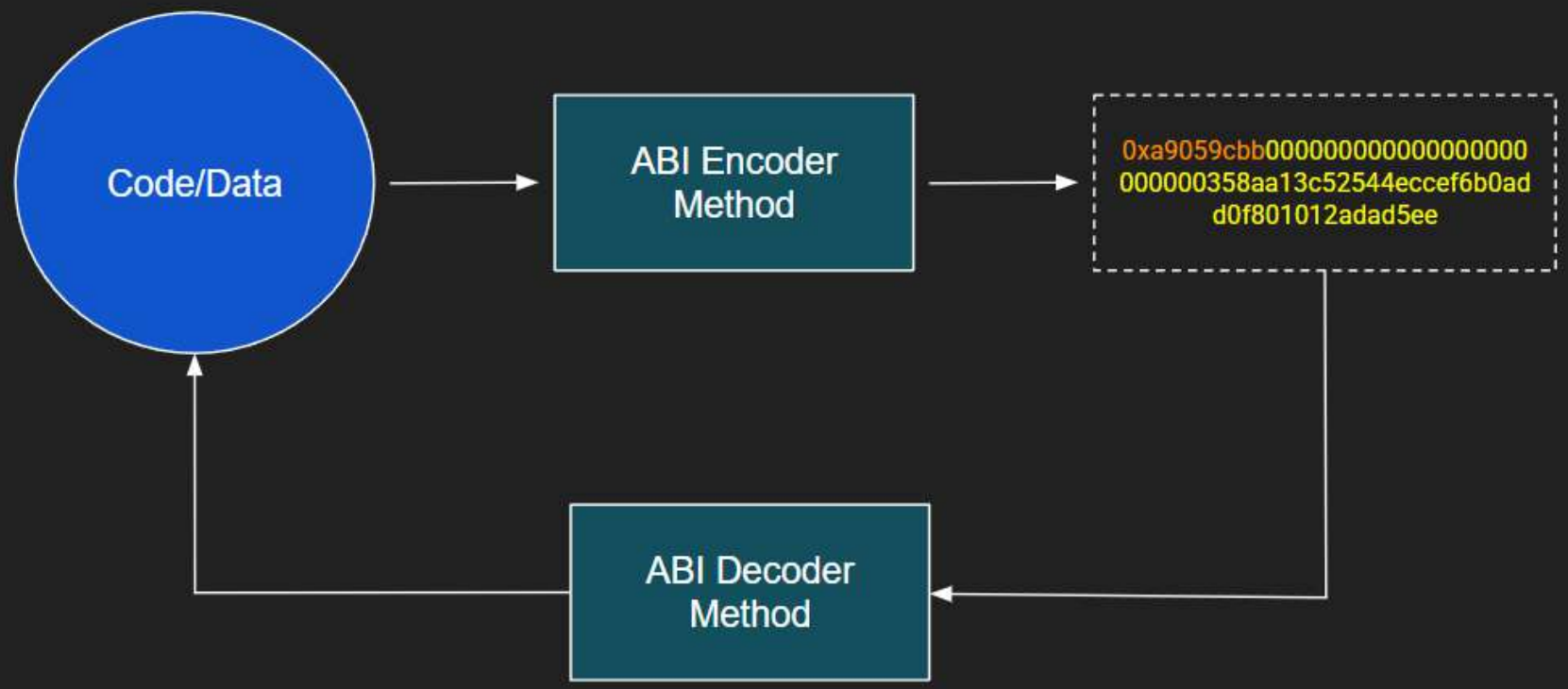
```
contract Bob {
    function setter(Alice _alice, uint256 _x) public {    undefined g
        _alice.setter(_x);
    }

    function getter(Alice _alice) public view returns(uint) {    un
       uint item =  _alice.getter();
       return item;
    }

    function setterViaAddress(address _addr, uint256 _x) public {
        Alice alice = Alice(_addr);
        alice.setter(_x);
    }

    function payableSetter(Alice _alice, uint256 _x) public payable
            _alice.payableSetter{value: msg.value}(_x);
    }
}
```

Caller Contract

```
contract Alice {
    uint256 public x;
    uint256 public value;

    function setter(uint256 _x) public {    2258
        x = _x;
    }

    function getter() public view returns(uint){
        return x;
    }

    function payableSetter(uint256 _x)    44653 g
        public
        payable
    {
        x = _x;
        value = msg.value;

    }
}
```

Callee Contract

ABI Encoding & Decoding



ABI Encoding & Decoding

ABI - ABI (Application Binary Interface) encoding and decoding in Solidity refer to the process of converting data between its native format (used within the Solidity code) and a format suitable for transmission and storage (used for contract interactions).

ABI Decoding - ABI decoding is the reverse process, where you take the encoded byte data and convert it back into its original parameters. This is useful when you receive data from a contract call or transaction and need to understand or use the original values.

ABI Encoding for Data Types

```
function encodeData(      infinite gas
    uint256 fixedNumber,
    string memory dynamicString,
    uint256[2] memory fixedArray,
    uint256[] memory dynamicArray) public pure returns (bytes memory) {
    return abi.encode(fixedNumber, dynamicString, fixedArray, dynamicArray);
}
```

2,abc,[1,2],[3,4]

Input

```
bytes: 0x0000000000000000000000000000000000000000000000000000000000000020
000000000000000000000000000000000000000000000000000000000000000a0000000000
0000000000000000000000000000000000000000000000000000000100000000000000000
0000000000000000000000000000000000000000000000000000000200000000000000000
000000000000000000000000000000000000000000000000000000e000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000
```

Understand The Encoding

Fixed Number (2)

0x0000000000000000000000000000000000000000000000000000000000000002

Pointer to Dynamic String

0x00000000000000000000000000000000000000000000000000000000000000a0

Fixed Array

0x0000000000000000000000000000000000000000000000000000000000000010
0x0000000000000000000000000000000000000000000000000000000000000001
0x0000000000000000000000000000000000000000000000000000000000000002

Pointer to Dynamic Array

0x00000000000000000000000000000000000000000000000000000000000000e0

Understand The Encoding

Dynamic String ("abc")

```
0x0000000000000000000000000000000000000000000000000000000000000003
0x6162630000000000000000000000000000000000000000000000000000000000
```

Dynamic Array

```
0x0000000000000000000000000000000000000000000000000000000000000002
0x0000000000000000000000000000000000000000000000000000000000000003
0x0000000000000000000000000000000000000000000000000000000000000004
```

ABI Encoding for Data Types

Assume we call `encodeData(42, "hello", [1, 2], [3, 4, 5])`.

fixedNumber (42)

- Directly encoded in a 32-byte segment.
- Hex: `0x000000000000000000000000000000000000000000000000000000000000002a`

dynamicString ("hello")

Then the length of the string (5 bytes):

`0x0000000000000000000000000000000000000000000000000000000000000005`

Followed by the UTF-8 encoded string "hello" padded to 32 bytes:

- `0x68656c6c6f000000000000000000000000000000000000000000000000000000`

ABI Encoding for Data Types

fixedArray ([1, 2])

- Encoded directly as two 32-byte segments.
- First element (1): `0x0000000000000000000000000000000000000000000000000000000000000001`
- Second element (2): `0x0000000000000000000000000000000000000000000000000000000000000002`

dynamicArray ([3, 4, 5])

Length of the array (3): `0x0000000000000000000000000000000000000000000000000000000000000003`
Followed by each element of the array padded to 32 bytes:

- First element (3): `0x0000000000000000000000000000000000000000000000000000000000000003`
- Second element (4): `0x0000000000000000000000000000000000000000000000000000000000000004`
- Third element (5): `0x0000000000000000000000000000000000000000000000000000000000000005`

ABI Encoding Rules

ABI encoding for fixed-size and dynamic-size data types in Ethereum follows different rules. The key difference lies in how dynamic types (such as strings, dynamic arrays, etc.) are encoded with additional information about their size and offset, whereas fixed-size types are encoded directly in a predictable, fixed manner.

Fixed-Size Types

Fixed-size types have a predefined size and are encoded directly in a single 32-byte (256-bit) word. This includes fixed-size arrays, integers, booleans, addresses, etc.

Dynamic-Size Types

Dynamic-size types do not have a predetermined size and require additional encoding information. They include dynamic arrays, strings, and bytes. The encoding for these types starts with a 32-byte word indicating the offset to the actual data, followed by the length of the data, and then the data itself.

Importance of ABI Encoding and Decoding

Inter-contract Communication: Contracts need a standardized way to communicate with each other. ABI encoding ensures that function calls are correctly understood by the receiving contract.

Off-chain Interactions: When interacting with contracts from off-chain applications (like a web3.js or ethers.js), ABI encoding allows these applications to send properly formatted data to the blockchain.

Data Integrity: Ensuring data is correctly formatted and understood is crucial for the integrity and security of smart contracts.

ABI Encoding For Call Data

Function	Syntax
encodeWithSignature	`abi.encodeWithSignature("functionSignature", [arg1, arg2, ...])`
encodeWithSelector	`abi.encodeWithSelector(functionSelector, to, amount)`
encodeCall	`abi.encodeCall(functionName, [arg1, arg2, ...])`

Encoding Call Data

```
function encodeWithSignature(address to, uint256 amount)    infinite gas
    external
    pure
    returns (bytes memory)
{
    return abi.encodeWithSignature("transfer(address,uint256)", to, amount);
}
```

Encode With Signature

```
function encodeWithSelector(address to, uint256 amount)    infinite gas
    external
    pure
    returns (bytes memory)
{
    return abi.encodeWithSelector(IERC20.transfer.selector, to, amount);
}
```

Encode With Selector

```
function encodeCall(address to, uint256 amount)    infinit
    external
    pure
    returns (bytes memory)
{
    return abi.encodeCall(IERC20.transfer, (to, amount));
}
```

Encode Call

ABI decoding

```
function decodeData(bytes memory data) public pure returns      infinite gas
    (
    uint256,
    string memory,
    uint256[2] memory,
    uint256[] memory
    ) {
        (
        uint256 fixedNumber,
        string memory dynamicString,
        uint256[2] memory fixedArray,
        uint256[] memory dynamicArray
        ) = abi.decode(data, (uint256, string, uint256[2], uint256[]));
        return (fixedNumber, dynamicString, fixedArray, dynamicArray);
}
```

Encoding Call Data - Output

`0xa9059cbb000000000000000000000000358aa13c52544eccef6b0add0f801012adad5ee3000000000000000000000000000000000000000000000000000000000000000a`

1. **Function Selector**: `0xa9059cbb`
   - This is the first 4 bytes of the Keccak-256 hash of the function signature `transfer(address,uint256)`. The function selector uniquely identifies the function in the smart contract.
2. **First Parameter (address)**: `000000000000000000000000358aa13c52544eccef6b0add0f801012adad5ee3`
   - The address is represented by the next 32 bytes. In this case, it's `0x358aa13c52544eccef6b0add0f801012adad5ee3`. The address is right-padded with zeros to fit into 32 bytes.
3. **Second Parameter (uint256)**: `000000000000000000000000000000000000000000000000000000000000000a`
   - The uint256 value is represented by the last 32 bytes. In this case, it's `0x0a`, which is 10 in decimal.

So, the decoded information is:

- **Function**: `transfer`
- **Parameters**:
  - **address**: `0x358aa13c52544eccef6b0add0f801012adad5ee3`
  - **uint256**: `10`

Abi Decoding

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

interface IERC20 {
    function transfer(address, uint256) external;    - gas
}
event TransferDecoded(address indexed to, uint256 amount);
contract Token {
    function transfer(address, uint256) external {    infinite gas
        // Extract the calldata excluding the function selector (first 4 bytes)
        bytes memory data = msg.data[4:];

        // Decode the parameters
        (address to, uint256 amount) = abi.decode(data, (address, uint256));

        // Now you can use the `to` and `amount` variables as needed
        // For demonstration, we'll just log the values (assuming an event is defined for logging)
        emit TransferDecoded(to, amount);
    }
}
```

Delegatecall

delegatecall is a low-level function in Solidity used for calling another contract's code within the context of the calling contract. It allows a contract to delegate its call to another contract (borrowing the functionality of another contract) while still preserving its own storage and context.

When a contract makes a delegatecall, the code at the target address is executed in the context of the calling contract. This means that the storage, state variables, and functions of the calling contract are accessible to the code being executed.

This means that the `msg.sender` and `msg.value` will remain unchanged, and the storage of the calling contract will be used instead of the called contract.

This allows for a form of code reuse while maintaining the state and context of the original contract.

Example

Calling Contract

Delegate call

Called Contract/Target Contract

call

`someNumber` of Called contract will be affected when setNumber() of CallerContract will be called.

```
contract CalledContract {
    uint public someNumber;
    function setNumber(uint _num) public {
        someNumber = _num;
    }
}
```

```
contract Caller {
    uint public someNumber;

    // Function to call setNumber on another contract
    function setNumber(address _calledContractAddress, uint _num) public {
        bytes memory data = abi.encodeWithSignature("setNumber(uint256)", _num);
        (bool success, ) = _calledContractAddress.call(data);
        require(success, "call failed");
    }
}
```

Delegatecall

In the the `someNumber` of Caller contract will be affected when setNumber() of CallerContract will be called.

```
contract CallerContract {
    uint public someNumber; // Note that this is the same variable name as in the Called Contract

    function setNumber(address _calledContractAddress, uint _num) public {    infinite gas
        // Prepare the data to call setNumber(uint) on the Called contract
        bytes memory data = abi.encodeWithSignature("setNumber(uint256)", _num);
        // Perform the delegatecall
        (bool success, ) = _calledContractAddress.delegatecall(data);
        require(success, "Delegatecall failed");
    }
}
```

```
contract CalledContract {
    uint public someNumber;
    function setNumber(uint _num) public {
        someNumber = _num;
    }
}
```

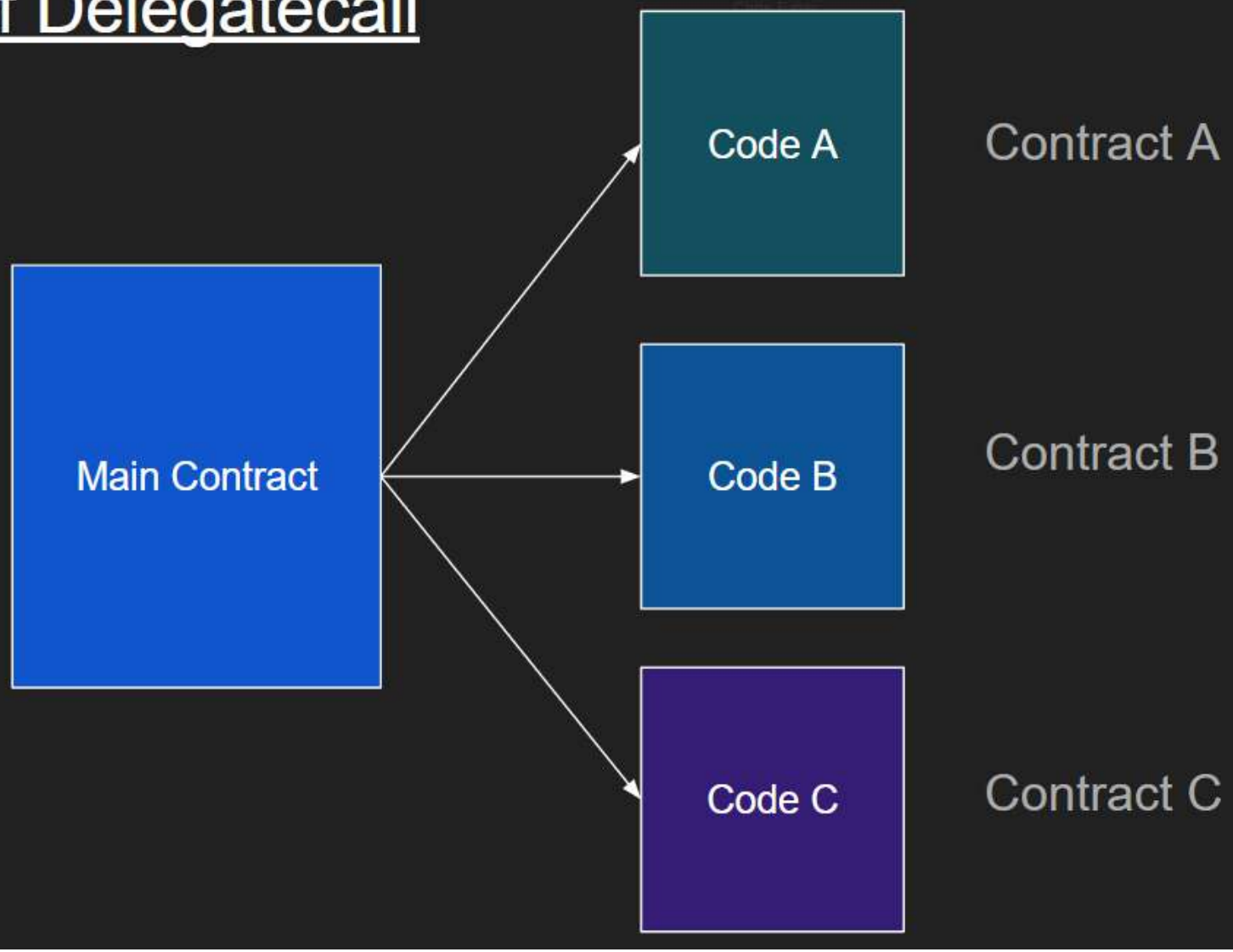
Delegate Call Vs Call

The primary distinction between call and delegateCall is how they handle the execution context of the function.

When you use **call**, the called function executes within the context of the calling contract. This means that the called function has access to the calling contract's storage and code.

On the other hand, when you use **delegateCall**, the called function executes in the context of the calling contract's caller. This means that the called function has access to the caller's storage and code, not the calling contract's storage and code.

Benefit of Delegatecall



Things To Note

All the state variable in the caller and called contrat should be same and declared in the same order. You can add state variables in the caller contract but without affecting the order of the variables that are same in both called and caller contract.

```
contract CallerContract {
    address public sender;
    uint public value;
```

```
contract CalledContract {
    address public sender;
    uint public value;
```

✔

```
contract CallerContract {
    address public sender;
    uint public value;
```

```
contract CalledContract {
    uint public value;
    address public sender;
```

✖ Variable Order Changed

```
contract CallerContract {
    address public sender;
    uint public value;
```

```
contract CalledContract {
    address public sender;
    uint public value;
    uint public num;
```

✔

```
contract CallerContract {
    address public sender;
    uint public value;
```

```
contract CalledContract {
    uint public num;
    address public sender;
    uint public value;
```

✖ Variable Order Changed as `num is declared before.

Things To Note

When ContractA uses `delegatecall` to invoke a function in ContractB, the execution context remains within ContractA. This implies that any state variable read or write operations will affect ContractA's state variables, not those of ContractB. Consequently, ContractB's state variables are isolated from ContractA's execution context and cannot be directly accessed or modified. However, ContractB's own state variables can still be accessed and altered through direct function calls to ContractB or by using `call` from another contract.

Benefits Of Using Delegatecall

Code Eater

Benefits of Using `delegatecall`

1. **Code Reuse**:
   - By using `delegatecall`, you can reuse common logic across multiple contracts without duplicating the code. This reduces deployment costs and makes the code easier to maintain.
2. **Upgradability**:
   - `delegatecall` is a key mechanism used in creating upgradable smart contract systems. With `delegatecall`, you can separate the contract logic (which can be upgraded) from the contract storage (which remains constant), allowing you to update or fix your contract logic without changing the contract state.
3. **Modularity**:
   - It allows for the separation of concerns by dividing a complex contract into smaller, more manageable parts. This can make the contract easier to understand and audit.

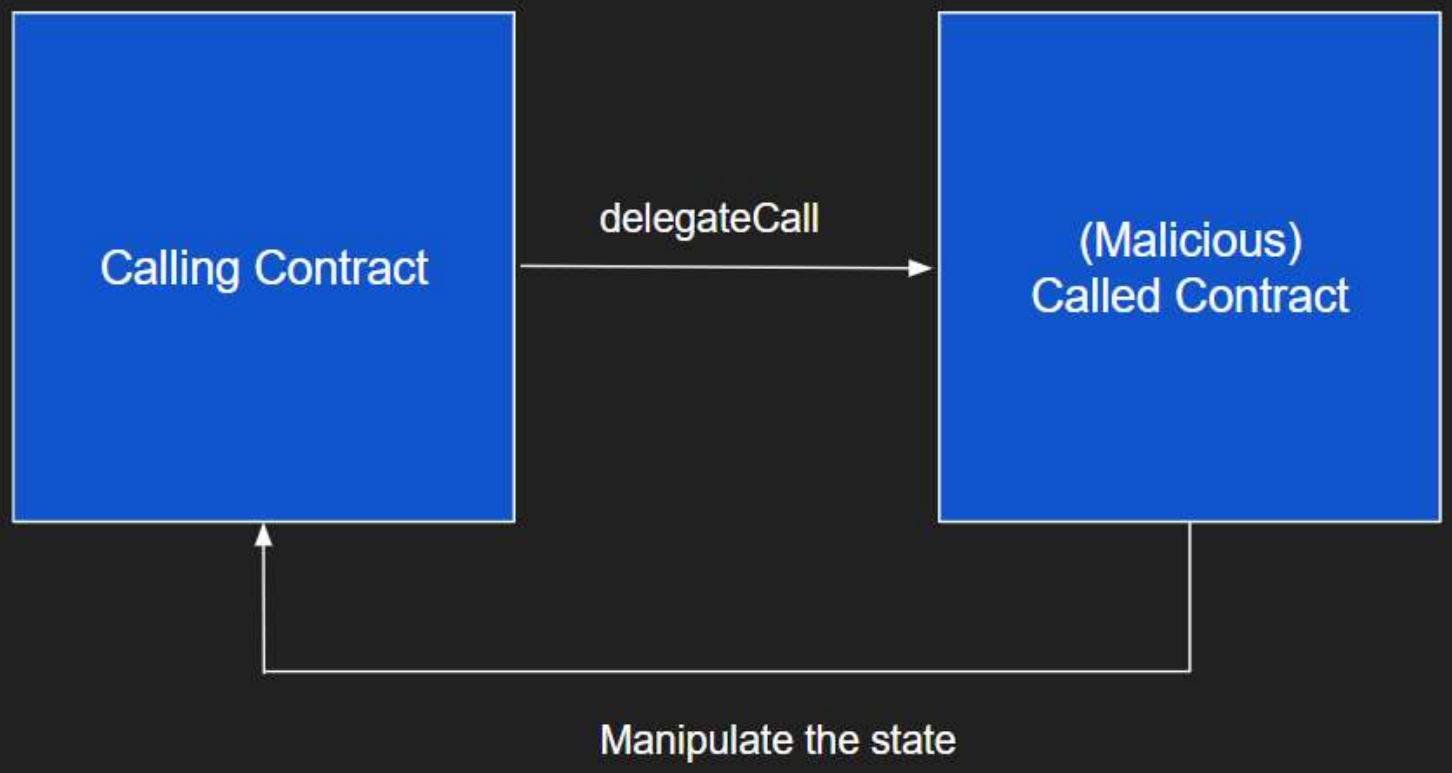
Code Eater

Delegatecall

Important Considerations

- **Security**: Using `delegatecall` can be risky if not properly controlled, as it can change the state of the calling contract in unpredictable ways. Ensure that the code being called is trusted.
- **Storage Layout**: Both the calling and called contracts should have the same storage layout for variables accessed by the called functions, as `delegatecall` uses the storage of the calling contract. Mismatched storage layouts can lead to unexpected behaviors.

Risk Of Delegate Call



Things To Note

```
contract A {
    function callB(address _bAddress, address _cAddress) public payable {
        // Call contract B with some Ether and pass the address of C
        (bool success, ) = _bAddress.call{value: msg.value}(
            abi.encodeWithSignature("delegateLog(address)", _cAddress)
        );
        require(success, "call to B failed");
    }
}
```

Call

```
contract B {
    address public sender;
    uint public value;
    function delegateLog(address _cAddress) public payable {
        sender = msg.sender;
        value = msg.value;
        bytes memory data = abi.encodeWithSignature("log()");
        (bool success, ) = _cAddress.delegatecall(data);
        require(success, "delegatecall failed");
    }
}
```

delegateCall

```
contract C {
    address public sender;
    uint public value;
    function log() public payable {
        sender = msg.sender;
        value = msg.value;
    }
}
```

sender = address of A
value = Ether sent by A

When a contract is called using delegatecall, the values of msg.sender and msg.value passed to the target contract are not those of the delegate call but instead those of the original message call to the calling contract.

This means that the target contract can't access the actual address of the account that sent the delegatecall or the amount of Ether sent with the delegate call.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract TransferExample {
    function transferEther(address payable recipient) public payable {
        require(
            address(this).balance >= 1 ether,
            "Insufficient contract balance"
        );
        recipient.transfer(1 ether);
    }

    function sendEther(address payable recipient)
        public
        payable
        returns (bool)
    {
        require(
            address(this).balance >= 1 ether,
            "Insufficient contract balance"
        );
        bool sent = recipient.send(1 ether);
        return sent;
    }

    function callEther(address payable recipient)
        public
        payable
        returns (bool, bytes memory)
    {
        require(
            address(this).balance >= 1 ether,
            "Insufficient contract balance"
        );
        (bool success, bytes memory data) = recipient.call{value: 1
ether}("");
        return (success, data);
    }

    // Allow contract to receive Ether
```

```
    // receive() external payable {}
}

/*
Difference Between transfer, send, and call
transfer

Gas Limit: 2300 gas
Returns Boolean? ❌ No
Revert on Failure? ✅ Yes (reverts if fails)
Security Risk: ✅ Safe (prevents reentrancy)
send

Gas Limit: 2300 gas
Returns Boolean? ✅ Yes (returns false on failure)
Revert on Failure? ❌ No
Security Risk: ❌ Less secure (must check return value manually)
call

Gas Limit: No limit (all available gas)
Returns Boolean? ✅ Yes (returns success & data)
Revert on Failure? ❌ No
Security Risk: ⚠️ High risk (prone to reentrancy attacks)
*/
```

CALLING ANOTHER CONTRACT(Here i can controll the alice contract threw bob contract)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract Alice {
    uint256 public x;
    uint256 public value;

    function setter(uint256 _x) public {
        x = _x;
    }
```

```
    function getter() public view returns (uint256) {
        return x;
    }

    function payableSetter(uint256 _x) public payable {
        x = _x;
        value = msg.value;
    }
}

contract Bob {
    function setter(Alice _alice, uint256 _x) public {
        _alice.setter(_x);
    }

    function getter(Alice _alice) public view returns (uint256) {
        uint256 item = _alice.getter();
        return item;
    }

    function setterViaAddress(address _addr, uint256 _x) public {
        Alice alice = Alice(_addr);
        alice.setter(_x);
    }

    function payableSetter(Alice _alice, uint256 _x) public payable {
        _alice.payableSetter{value: msg.value}(_x);
    }
}
```

ABI encoding & decoding

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract enocdedata {
    function encodeData(
        uint256 fixedNumber,
        string memory dynamicString,
        uint256[2] memory fixedArray,
        uint256[] memory dynamicArray
    ) public pure returns (bytes memory) {
        return abi.encode(fixedNumber, dynamicString, fixedArray,
dynamicArray);
    }

    function decodeData(bytes memory data)
        public
        pure
        returns (
            uint256,
            string memory,
            uint256[2] memory,
            uint256[] memory
        )
    {
        (
            uint256 fixedNumber,
            string memory dynamicString,
            uint256[2] memory fixedArray,
            uint256[] memory dynamicArray
        ) = abi.decode(data, (uint256, string, uint256[2], uint256[]));
        return (fixedNumber, dynamicString, fixedArray, dynamicArray);
    }
}
```

ABI encoding and decoding of calldata

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

interface IERC20 {
    function transfer(address recipient, uint256 amount) external;
}

contract Token is IERC20 {
    event Transfer(address recipient, uint256 amount);

    function transfer(address _recipient, uint256 _amount) external
override {
        emit Transfer(_recipient, _amount);
    }
}

contract AbiEncode {
    function callTokenContractFunction(address _contract, bytes calldata
data)
        external
    {
        (bool ok, ) = _contract.call(data);
        require(ok, "transaction failed");
        // Optionally handle returnedData if needed
    }

    function encodeWithSignature(address to, uint256 amount)
        external
        pure
        returns (bytes memory)
    {
        return abi.encodeWithSignature("transfer(address,uint256)", to,
amount);
    }

    function encodeWithSelector(address to, uint256 amount)
        external
        pure
        returns (bytes memory)
```

```
    {
        return abi.encodeWithSelector(IERC20.transfer.selector, to,
amount);
    }

    function encodeCall(address to, uint256 amount)
        external
        pure
        returns (bytes memory)
    {
        return abi.encodeCall(IERC20.transfer, (to, amount));
    }
}


// 🚀 Purpose of the Contract
// The contract AbiEncode provides functions to:

// Encode function calls in different ways (encodeWithSignature,
encodeWithSelector, encodeCall).
// Send encoded data to another contract (callTokenContractFunction).
// This is useful when interacting with external contracts dynamically
without directly importing their interfaces.
```

DELEGATECALL(1)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract CalledContract { //painter
    uint256 public someNumber;

    // Function to set the value of someNumber
    function setNumber(uint256 _num) public {
        someNumber = _num;
    }
}

contract Caller { //owner
```

```
    uint256 public someNumber; // Note that this is the same variable name
as in the Called Contract

    // Function to call setNumber on another contract
    function setNumber(address _calledContractAddress, uint256 _num)
public {
        // Prepare the data to call setNumber(uint) on the Called contract
        bytes memory data = abi.encodeWithSignature("setNumber(uint256)",
_num);

        // Perform the call
        (bool success, ) = _calledContractAddress.call(data);

        // Check if the call was successful
        require(success, "call failed");
    }
}
```

DELEGATECALL(2)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract CalledContract {//painter
    uint256 public someNumber;

    function setNumber(uint256 _num) public {
        someNumber = _num;
    }
}

contract Caller {//owner
    uint256 public someNumber; // Note that this is the same variable name
as in the Called Contract

    function setNumber(address _calledContractAddress, uint256 _num)
public {
        // Prepare the data to call setNumber(uint) on the Called contract
        bytes memory data = abi.encodeWithSignature("setNumber(uint256)",
_num);
```

```
        // Perform the delegatecall
        (bool success, ) = _calledContractAddress.delegatecall(data);
        require(success, "Delegatecall failed");
    }
}
//painter ne kya kiya --- mere resources se mere hi ghar me kam kiya.yani
ki mera storage or mere me hi changes
```